

第5章

时钟系统与休眠模式



第5章 时钟系统与休眠模式

- 5.1 时钟系统

- 5.2 休眠模式

5.2 休眠模式



- 5.2.1 休眠模式与低功耗
- 5.2.2 休眠唤醒与退出

5.2.1 休眠模式与低功耗



- MSP430之所以能够做到极低的功耗，
 - ◆ 一方面是其采用了较为先进的集成电路设计工艺与理念。
 - ◆ 另一方面**与其独特的时钟系统是分不开的。**
- 功耗与CPU工作频率有密切关系。
- MSP430的时钟系统可以产生MCLK、SMCLK、ACLK三路相互独立的时钟信号。
- 这使通过关闭不用的功能模块以降低功耗的方法成为可能。
- 实际上，关闭模块最有效的方法就是关闭该模块的时钟。
- MSP430就是通过该方法来实现低功耗功能的。

5.2.1 休眠模式与低功耗

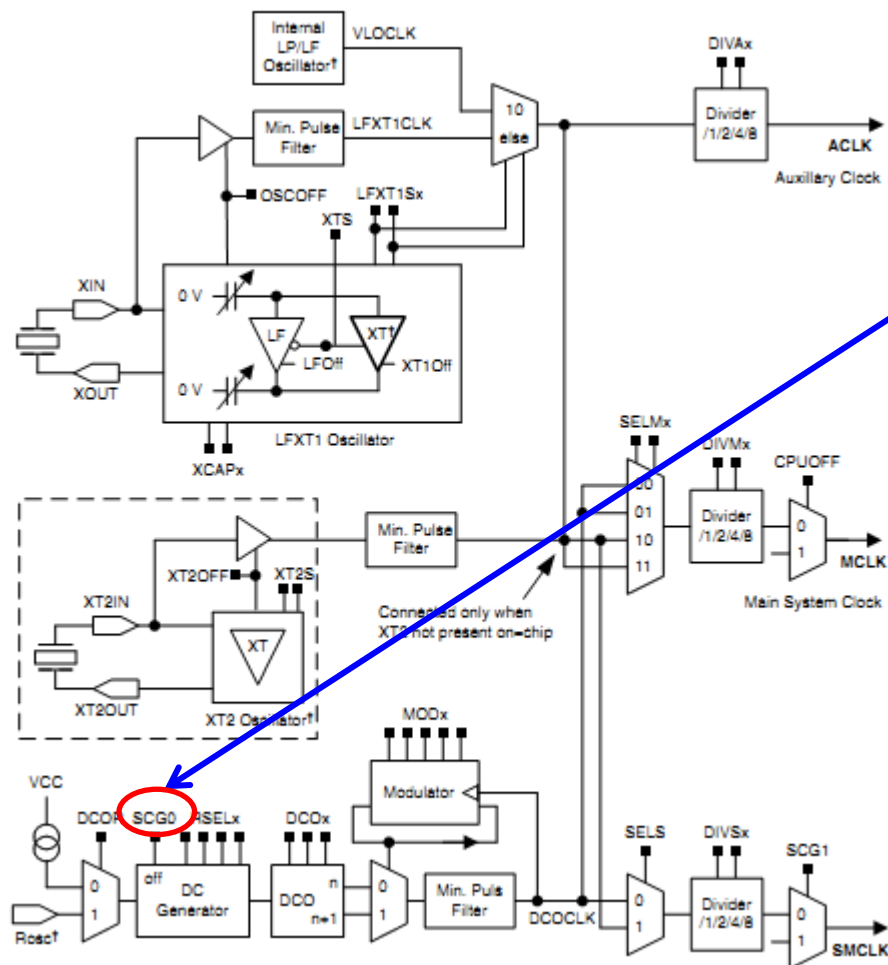


- MSP430F261x系列单片机共有6种工作模式，其中一个**是活动模式**，其余5种为**低功耗模式**。低功耗模式（LPM）也称为**休眠模式**。
- 这些模式分别由**状态寄存器中4个状态控制位**进行控制。这4个控制位的不同组合对应单片机6种不同的工作方式。

5.2.2 休眠唤醒与退出



Figure 5-1. Basic Clock Module+ Block Diagram



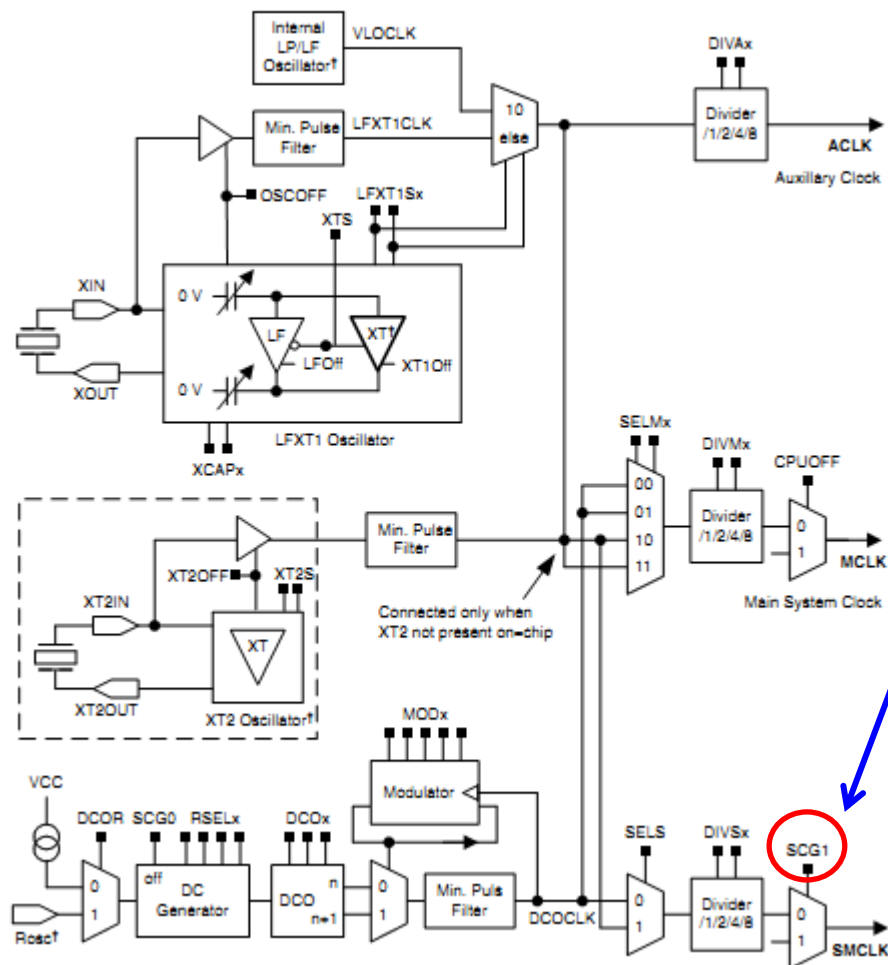
控制位的影响

- **SCG0:**
 - ◆ 控制直流发生器的工作。
 - ◆ 当SCG0被置位时禁止，复位时激活。

5.2.2 休眠唤醒与退出



Figure 5-1. Basic Clock Module+ Block Diagram



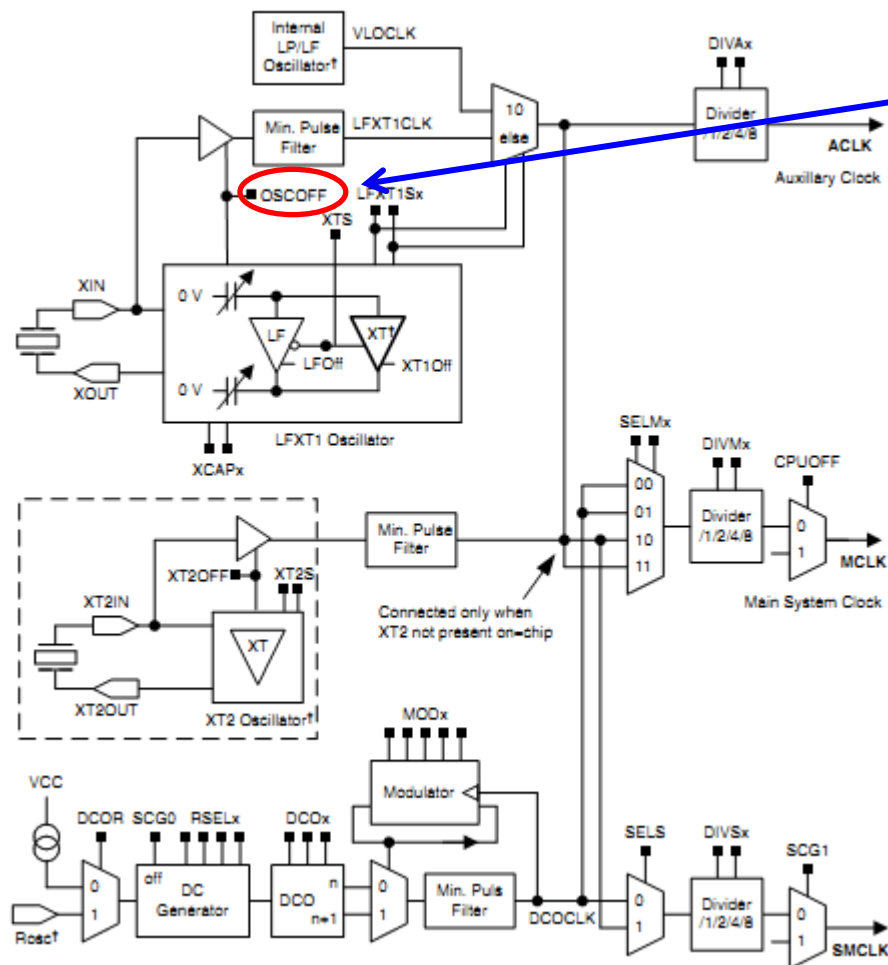
● SCG1:

- ◆ 当SCG1复位时，使能SMCLK；
- ◆ SCG1置位则禁止SMCLK。

5.2.2 休眠唤醒与退出



Figure 5-1. Basic Clock Module+ Block Diagram



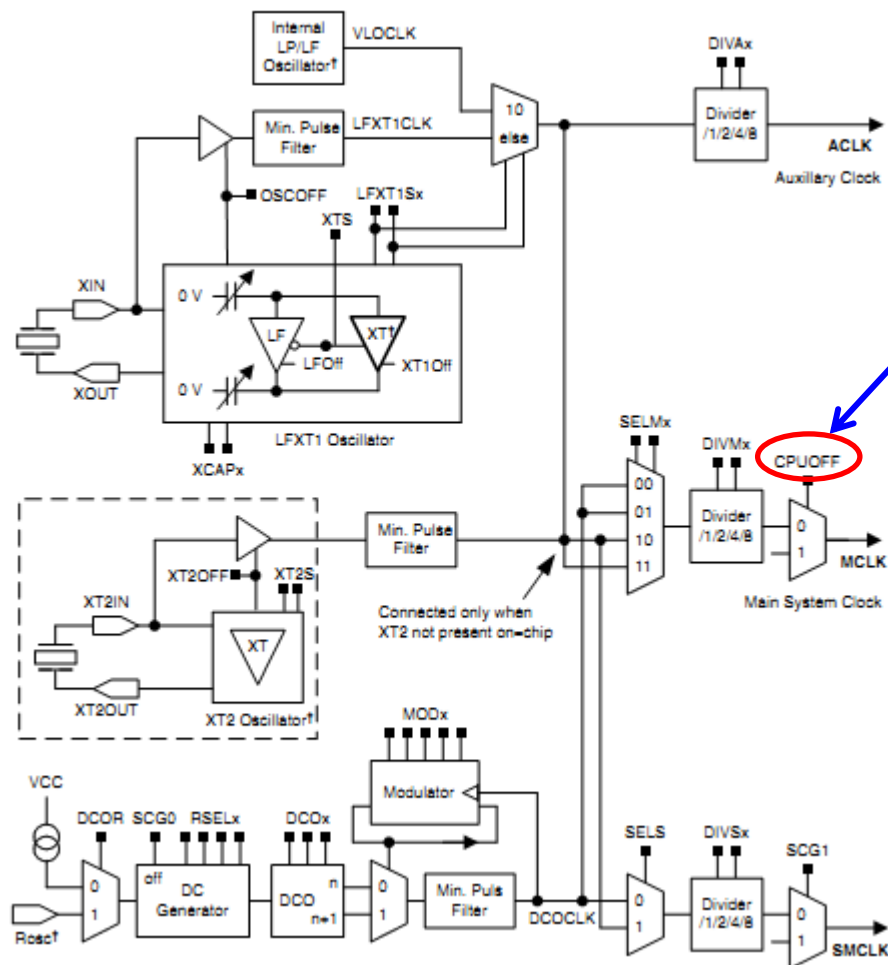
● OSCOFF:

- ◆ 当OSCOFF复位时，LFXT1晶体振荡器被激活。
- ◆ 当OSCOFF被置位时禁止。

5.2.2 休眠唤醒与退出



Figure 5-1. Basic Clock Module+ Block Diagram



● CPUOFF:

- ◆ 当CPUOFF复位时，用于CPU的时钟信号MCLK被激活；
- ◆ 当CPUOFF置位时，MCLK停止

5.2.1 休眠模式与低功耗



时钟工作模式	MCLK	SMCLK	ACLK
活动模式 (ACTIVE)	开	开	开
低功耗模式0 (LPM0)	关	开	开
低功耗模式1 (LPM1)	关	开	开
低功耗模式2 (LPM2)	关	关	开
低功耗模式3 (LPM3)	关	关	开
低功耗模式4 (LPM4)	关	关	关

5.2.1 休眠模式与低功耗



- **活动模式：**

- ◆ 单片机的正常工作状态，即所有时钟全部启动。
- ◆ 该模式下单片机功耗最大。

- **低功耗模式0：**

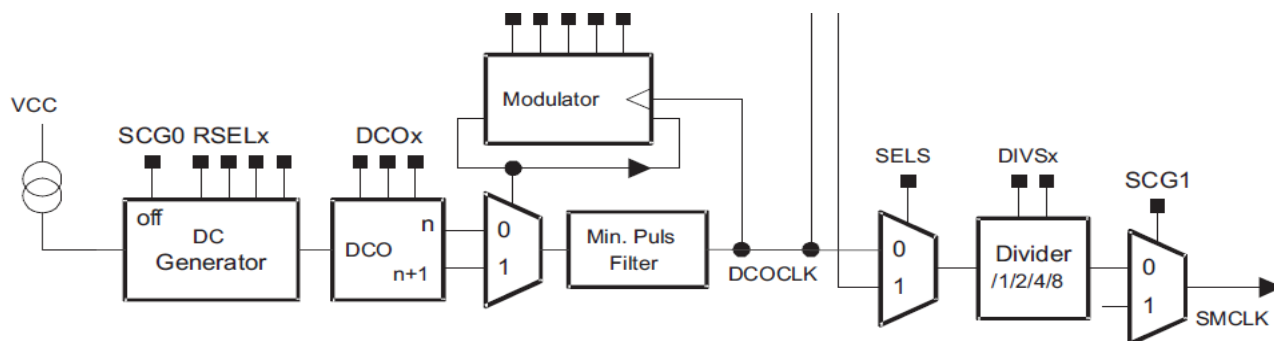
- ◆ 该模式下，时钟系统仍处于正常工作状态，与活动模式的区别是**MCLK的输出通道被关闭**，**CPU无法工作**。其他时钟均是可用的。

5.2.1 休眠模式与低功耗



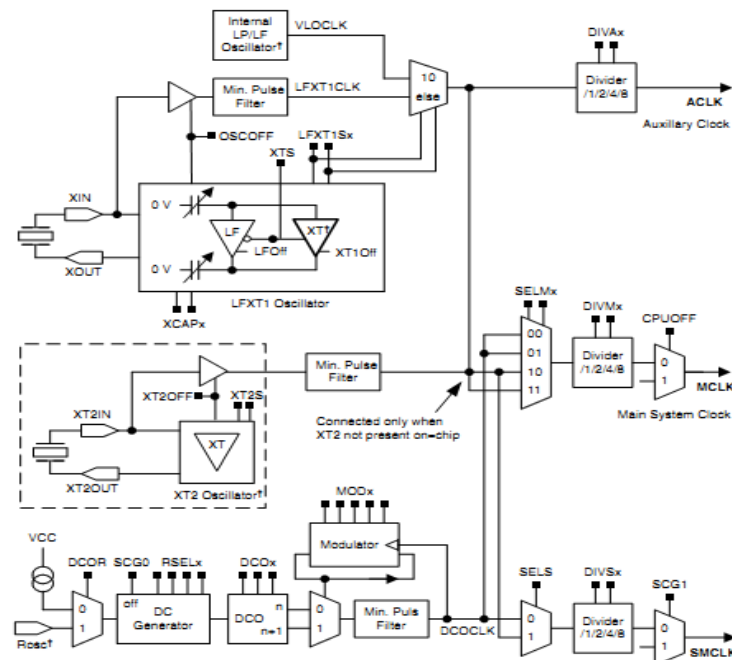
● 低功耗模式1:

- ◆ 该模式下，MCLK与直流发生器被关闭。
- ◆ 直流发生器一旦关闭，意味着DCO就不能正常工作，即SMCLK无法使用DCOCLK。
- ◆ 但SMCLK仍可以作为时钟使用。



5.2.1 休眠模式与低功耗

Figure 5-1. Basic Clock Module+ Block Diagram



● 低功耗模式2:

◆ 该模式下，**MCLK与SMCLK被关闭**，但**直流发生器工作**，即**DCOCLK可用**。

◆ 由于SMCLK与MCLK均被关闭，所以DCOCLK可用的意义不大。

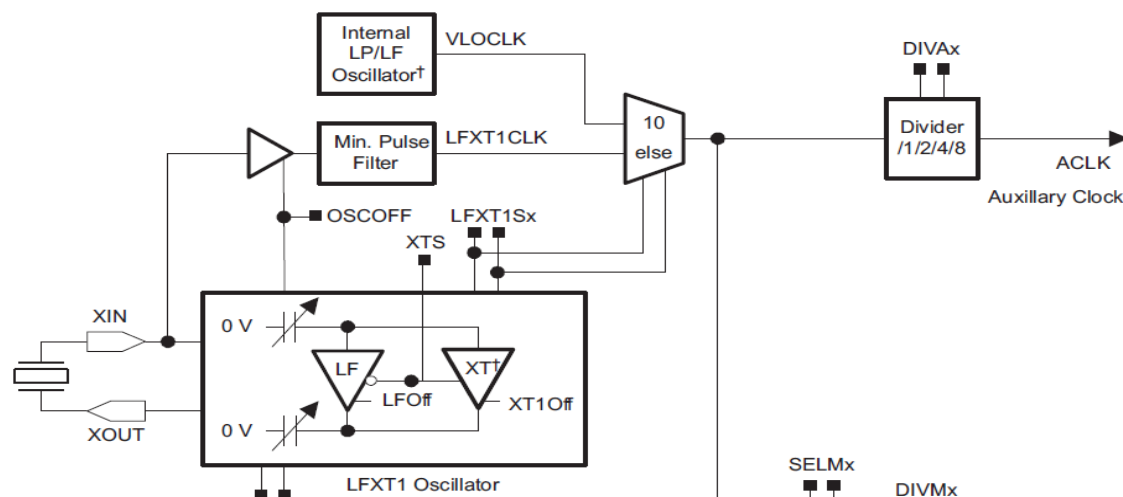
◆ 因此该模式不经常使用，**一般由低功耗模式3代替**。

5.2.1 休眠模式与低功耗



● 低功耗模式3:

- ◆ 该模式下，**MCLK、SMCLK与直流发生器均被关闭**，只有低频振荡器处于工作状态。
- ◆ 在该模式下**只有将ACLK作为时钟源的模块能够正常**。另外由于该模式下功耗较小，**且ACLK时钟可用于唤醒CPU**，故该模式最为常用。



5.2.1 休眠模式与低功耗



● 低功耗模式4:

- ◆ 该模式下，整个时钟系统被全部关闭。单片机内的所有部件均停止工作。
- ◆ 当然这时的功耗也就降到了最小。
- ◆ 该模式下，单片机无法由内部模块唤醒，只能通过外部中断或复位操作将系统唤醒。

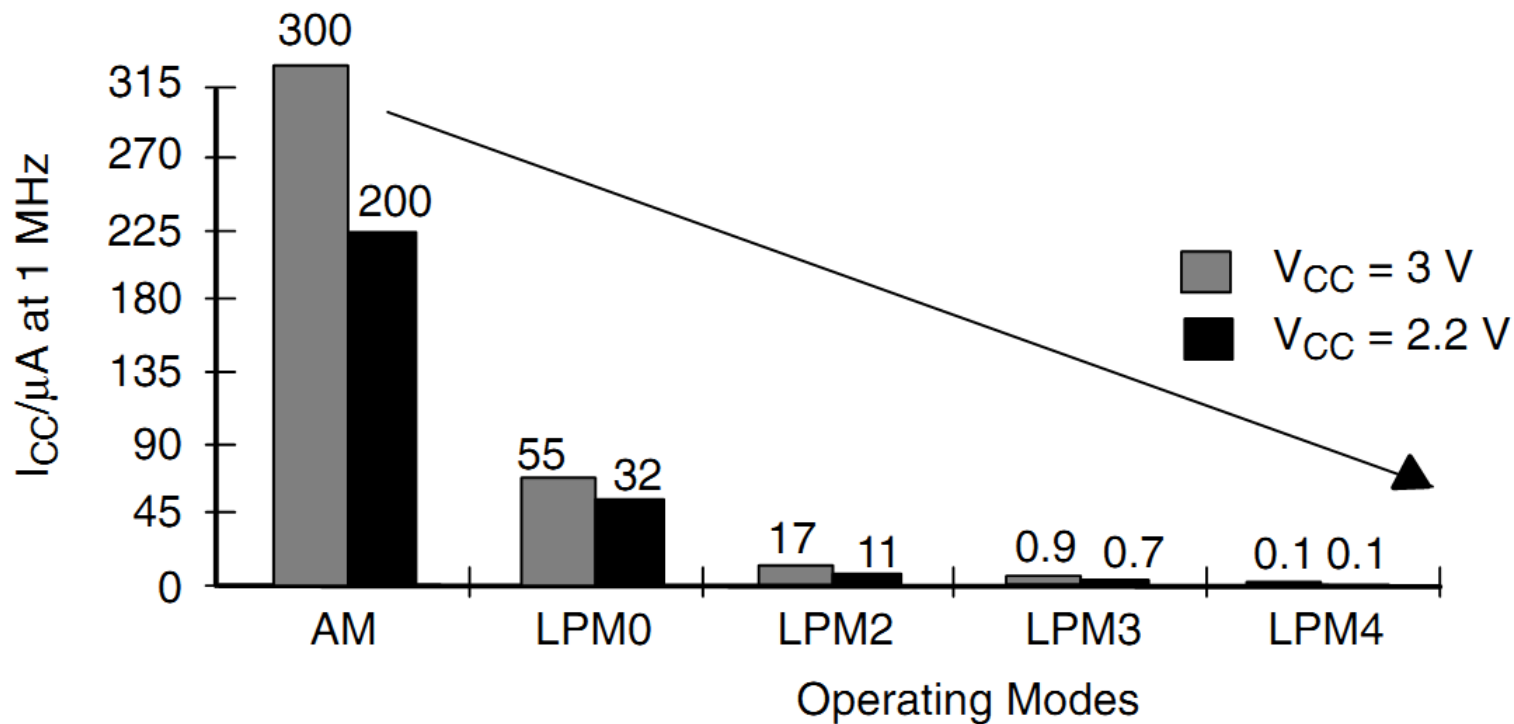
注意：复位操作唤醒系统意味着整个程序将重新开始执行。故该模式下的复位操作常用来实现“软关机”



5.2.1 休眠模式与低功耗

部件/控制位 工作模式	子系统 时钟	直流 发生器	低频 振荡器	主系统 时钟	功耗 (Vcc=3.0V, 25°C LFXT1=32.768KHz)
	SCG1	SCG0	OSCOFF	CPUOFF	
活动模式 (ACTIVE)	0(开)	0(开)	0(开)	0(开)	515μA/MHz
低功耗模式0 (LPM0)	0(开)	0(开)	0(开)	1(关)	87 μA
低功耗模式1 (LPM1)	0(开)	1(关)	0(开)	1(关)	40 μA
低功耗模式2 (LPM2)	1(关)	0(开)	0(开)	1(关)	25 μA
低功耗模式3 (LPM3)	1(关)	1(关)	0(开)	1(关)	1.1 μA
低功耗模式4 (LPM4)	1(关)	1(关)	1(关)	1(关)	0.2 μA

各个模式的功耗示意图



5.2 休眠模式



- 5.2.1 休眠模式与低功耗
- 5.1.2 休眠唤醒与退出

5.2.2 休眠唤醒与退出



- 单片机一旦进入休眠模式，**只有通过中断机制才能将其唤醒**，进入正常活动状态。
- 根据中断响应的原理，当中断请求得到响应后，系统会自动将状态寄存器的内容压入堆栈中，然后**将状态寄存器清零**。单片机随即退出休眠模式，进入正常的活动模式。
- 接着转而处理中断服务程序。

Figure 3-6. Status Register Bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							V	SCG1	SCG0	OSC OFF	CPU OFF	GIE	N	Z	C

5.2.2 休眠唤醒与退出



- 当执行完中断服务程序以后，CPU将自动地从堆栈中恢复状态寄存器原来的值。
- 由于状态寄存器中保存着原先的休眠模式，故系统又将进入原先的休眠模式。
- 因此，这种休眠唤醒机制较适合那些在中断内处理全部任务的应用，即一旦执行完中断服务程序后便立即进入休眠模式，等待下一个任务。这样就可以让休眠代替CPU的等待。

5.2.2 休眠唤醒与退出



- 若想从休眠模式唤醒后，使系统进入正常的活动模式或其他休眠模式，
- 需要在中断服务程序结束前修改堆栈内状态寄存器的值。
- 具体的做法是（以C语言为例）

5.2.2 休眠唤醒与退出



能够在中断服务程序里使用的函数

(1) **unsigned short __get_SR_register_on_exit(void)**: 用于中断函数返回时，返回状态寄存器SR的值。该函数只在中断函数中有效。

(2) **void __bic_SR_register_on_exit(unsigned short)**: 用于中断函数返回时，将状态寄存器SR中的某些位清0。输入参数为掩膜位，即需要清零的位为1，其他位为0。该函数无返回值，只能用于中断函数中。

简写为: **_BIC_SR_IRQ(unsigned short)**;

如:

```
__bic_SR_register_on_exit(LPM3_bits); //退出低功耗模式3，进入正常模式
```

或 **_BIC_SR_IRQ(LPM3_bits);**

5.2.2 休眠唤醒与退出



(3) **void __bis_SR_register_on_exit(unsigned short):** 用于中断函数返回时，将状态寄存器SR中的某些位置1。输入参数为掩膜位，即需要置位的位为1，其他位为0. 该函数无返回值，只能用于中断函数中。

简写为: **_BIS_SR_IRQ(unsigned short);**

如:

__bis_SR_register_on_exit(LPM4_bits); //退出中断程序后，进入低功耗模式4

或 **_BIS_SR_IRQ(LPM4_bits);**

5.2.2 休眠唤醒与退出



为方便起见，还可使用下面的宏指令：

- **LPM0_EXIT**; //退出LPM0，同时清除相关控制位
- **LPM1_EXIT**; //退出LPM1，同时清除相关控制位
- **LPM2_EXIT**; //退出LPM2，同时清除相关控制位
- **LPM3_EXIT**; //退出LPM3，同时清除相关控制位
- **LPM4_EXIT**; //退出LPM4，同时清除相关控制位
- **__low_power_mode_off_on_exit()**; //从任一休眠模式退出

5.2.2 休眠唤醒与退出



进入休眠模式，可用下面的宏指令：

- **__low_power_mode_0()**或**LPM0**; //进入LPM0
- **__low_power_mode_1()**或**LPM1**; //进入LPM1
- **__low_power_mode_2()**或**LPM2**; //进入LPM2
- **__low_power_mode_3()**或**LPM3**; //进入LPM3
- **__low_power_mode_4()**或**LPM4**; //进入LPM4

5.2.2 休眠唤醒与退出



模式转换关系

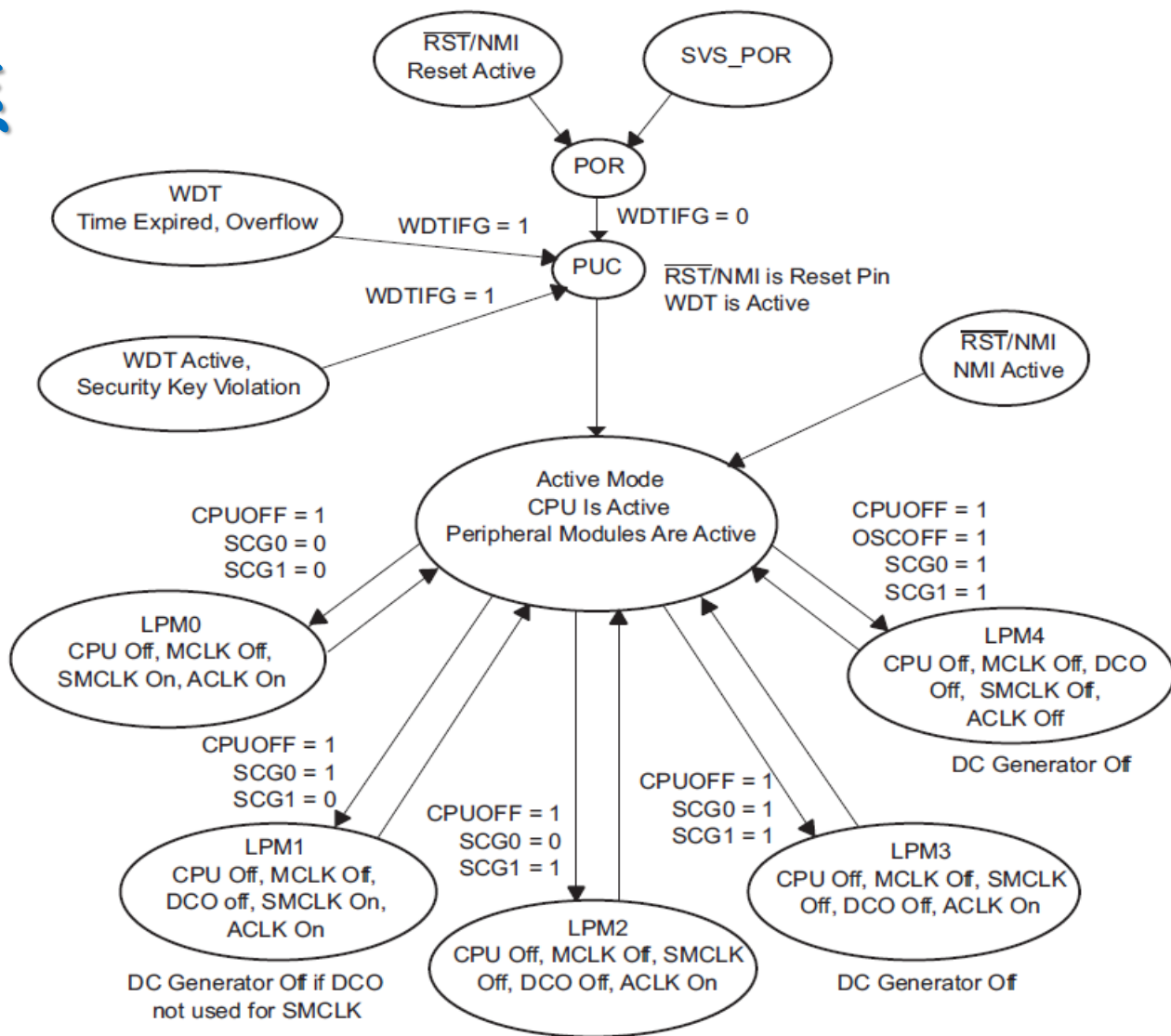


Figure 2-9. Operating Modes For Basic Clock System

5.2.2 休眠唤醒与退出



- 处于休眠状态的MSP430单片机通常工作在“休眠-唤醒-休眠”的循环中。
- 该方式可以让休眠代替CPU的等待，进而一方面减轻了CPU负担，另一方面也极大的降低了系统功耗。

5.2.2 休眠唤醒与退出



例：如下连接，使P1.0处的LED按照一定周期不停地闪烁。

软件延时方式

```
# include "msp430.h"
```

```
void main(void)
```

```
{
```

```
WDTCTL = WDTPW + WDTHOLD;
```

//关看门狗

```
P1DIR |= BIT0;
```

//设置P1.0为输出

```
while(1)
```

```
{
```

```
    P1OUT ^= BIT0;
```

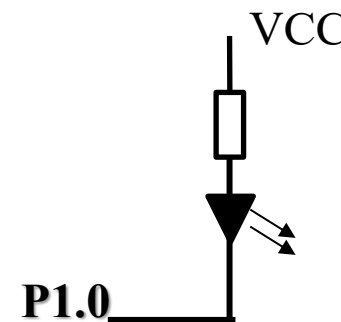
//取反

```
    for ( unsigned int i=0; i<20000; i++);
```

//延时

```
}
```

```
}
```



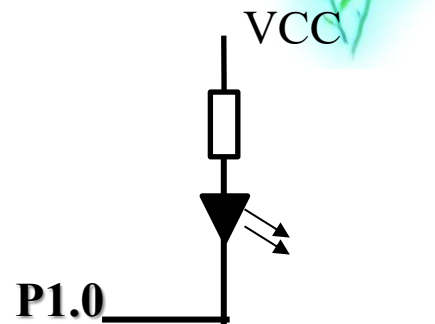
5.2.2 休眠唤醒与退出

中断方式

```
#include "msp430.h"
void main(void)
{
    WDTCTL = WDT_MDLY_32;           //设置定时长度及时钟源; 32ms
    IE1 |= WDTIE;                   //打开WDT中断
    P1DIR |= BIT0;
    __bis_SR_register(LPM0_bits + GIE); //GIE置位, 进入LPM0
}

#pragma vector = WDT_VECTOR          //WDT中断服务程序
__interrupt void WDT_ISR(void)
{
    P1OUT ^= BIT0;                   //取反
}

#define WDT_MDLY_32 (WDTPW+WDTTMSEL+WDTCNTCL) /* 32ms*/
```



5.2.2 休眠唤醒与退出



261x.h 头文件中的宏定义

```
#define GIE          (0x0008)
#define CPUOFF       (0x0010)
#define OSCOFF       (0x0020)
#define SCG0         (0x0040)
#define SCG1         (0x0080)

#define LPM0_bits     (CPUOFF)
#define LPM1_bits     (CPUOFF + SCG0)
#define LPM2_bits     (CPUOFF + SCG1)
#define LPM3_bits     (CPUOFF + SCG0 + SCG1)
#define LPM4_bits     (CPUOFF + SCG0 + SCG1 + OSCOFF)
```

5.2.2 休眠唤醒与退出

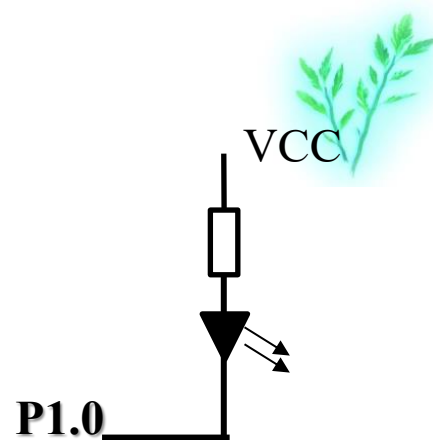


中断服务程序过长，会影响整个系统的中断响应。因此，在复杂程序设计中，应尽量减少中断服务程序的处理时间。

通常的做法：将原本放在中断服务程序中的任务放在主程序中，中断服务函数只负责给主程序传递一个启动主程序的信号。

5.2.2 休眠唤醒与退出

改进的中断方式



```
#include "msp430.h"
void main(void)
{
    WDTCTL = WDT_MDLY_32;           //设置定时长度及时钟源; 32ms
    IE1 |= WDTIE;                   //打开WDT中断
    P1DIR |= BIT0;
    while(1)
    {
        __bis_SR_register (LPM0_bits + GIE); //GIE置位, 进入LPM0
        P1OUT ^= BIT0;                  //取反
    }

    #pragma vector = WDT_VECTOR      //WDT中断服务程序
    __interrupt void WDT_ISR(void)
    {
        __low_power_mode_off_on_exit(); //从任一休眠模式退出
    }
}
```